The Inference Identification Process

Purpose and Scope

Inference maintains the protocol model stored in the database. After a game patch invalidates that model, Inference observes captured traffic and heals the model: First by re-identifying which wire opcode carries which known message type, later by repairing mutated field layouts. This document covers the identification phase only. Layout healing is a separate, harder problem, predicated on identification, and is deferred. Inference is distinct from Glass: Glass consumes the protocol model; Inference maintains it.

Inputs and the Unit of Assessment

The process begins with messages already collected. Extracted from the network stream, not raw packets (from Wireshark, Glass, or Inference captures). Each stored message carries metadata: session/character, arrival time, direction, and is extensible. Captures are tagged with which characters were logged in.

Individual messages are never scored in isolation. Messages are aggregated by wire opcode, and the collection is the unit of assessment. Handlers are free to partition a collection further (per session, per time window) when a feature requires it. Ordering evidence in particular is only meaningful within a single client's stream, since a multi-client capture interleaves several conversations.

Trust Model and Change Model

After a patch, nothing in the existing model is trusted. Every stored definition is a hypothesis to be tested, never an assumption.

The empirical change model, from history: A patch performs (1) a near-total relabeling of the 16-bit wire opcode space (effectively a permutation), with layouts untouched for the large majority of message types, and (2) for a small set of messages (typically 2–3 per patch, position updates being repeat targets), a permutation of the fields within the message: Identical fields at identical bit widths, reordered, with padding repacked, occasionally with a field appended. No field resizes or encoding changes have been observed. The conservation law: The field inventory is invariant even when layout is not. This is the strongest structural prior available, and it is what will eventually make healing

tractable. Whether the opcode relabeling is strictly total (no opcode ever survives at its old value) is unconfirmed; handlers may test it.

Statistical Framework

Identification is Bayesian hypothesis testing, implemented as a naive Bayes classifier. Each handler embodies one hypothesis H: e.g. "wire opcode 0x917C is NPCMoveUpdate", and applies a set of rules. Each rule is a feature with a likelihood function answering $P(observation|H)$ vs $P(observation|\neg H)$.

The ratio is the ***likelihood ratio***: Its logarithm is the weight of evidence (I. J. Good's term; Good and Turing measured it in decibans at Bletchley). Summing weights across features is valid exactly when the features are conditionally independent given H (the "naive" assumption). The complete update is: $prior\ odds + summed\ weight\ of\ evidence = \boldsymbol{posterior\ odds}$, and the reported confidence is the posterior converted back from log-odds.

Handlers report each feature's contribution in decibans (10 db = 10:1 odds, 20 db = 100:1), so a recommendation's evidence summary reads as an itemized ledger.

Scoring is one-vs-rest: a handler estimates only the likelihood that a collection is its own message type; ¬H is "something else." Handlers do not propose alternate identifications, but handlers may disagree, and because the true opcode mapping is a bijection, the ensemble of claims is formally a bipartite assignment problem (message types × wire opcodes, one-to-one). Conflicting strong claims on the same wire opcode are informative and must surface as conflicts.

Strength and Weight of Evidence

Every piece of evidence carries two distinct measures.

Strength is the balance of the evidence: the likelihood ratio a feature contributes, reported in decibans. It answers "which way does this evidence point, and how far?" Strength is what the ledger itemizes and what the posterior is built from.

Weight is the amount of data standing behind that likelihood ratio: how many independent contexts (captures) were used to fit the feature's model. It answers "how much would this assessment move if we saw one more capture?" A feature fitted from one capture has low weight however strong its ratio looks. A feature whose behavior has been confirmed across twenty captures has high weight.

The terms come from the literature on evidential reasoning (Keynes distinguished the balance from the weight of an argument; the modern phrasing is strength of evidence versus weight of evidence).

Weight enters the arithmetic through the hierarchical treatment. A feature's parameters are not constants. They are estimated from per-capture measurements, and the uncertainty of that estimate is carried into the likelihood as a predictive distribution (for the proportion feature, a Beta over the expected share, giving a Beta-binomial). Low weight widens the predictive distribution and automatically weakens the decibans the feature can produce. High weight tightens it. No importance factors are assigned by hand; a feature's influence is a measured consequence of its reliability.

Two rules follow:

First, the statistic-selection rule. Different statistics of the same underlying data can carry very different weight. Traffic share is a high-variance statistic across capture contexts. Rank in the frequency ordering is a low-variance statistic of the same counts; for a dominant opcode it may be nearly invariant across every adequate capture. When two candidate features are statistics of the same measurement, a handler uses the one with the higher measured weight, and never both, because that would count one measurement twice.

Second, the baseline-store rule. The database of prior-patch baselines must hold per-capture measurements (share, size histogram, and any other fitted statistic, recorded per opcode per analyzed capture), not single summary numbers. Hyperparameters are fitted from these measurements, so every capture analyzed adds weight to the features it touches, and the system's feature importances improve from its own history.

Evidence Scaling

When observations are conditionally independent given the hypothesis, the total weight of evidence is the sum of per-message log-likelihood ratios, so evidence grows with n. Formally, expected evidence accrues at rate $n \times D_{KL}$, the Kullback–Leibler divergence between what the hypothesis predicts and what the alternative predicts. This is why small captures are inherently unreliable without any special rule: the posterior simply stays near the prior. It only works if features are encoded per-observation: "rank #1" as a binary feature carries the same weight at n = 500 as at n = 65,738, which is wrong; the count itself, under a binomial (or Beta-binomial) likelihood, scales correctly and makes rank a consequence rather than a feature.

Messages in a capture are not i.i.d.: One zone with many NPCs floods correlated movement updates, so naive summation overstates confidence. The standard corrections are effective sample size (n_eff < n) or aggregation to a coarser unit before scoring (per time window, per entity). Miscalibration is the symptom that this correction is insufficient.

Feature Taxonomy

Features fall into two classes. Protocol-invariant features are properties of the protocol itself and are stable across captures. Direction, size distribution, field structure, and layout fit belong to this class. Behavior-dependent features are properties of what the players happened to be doing during the capture. Traffic share and timing belong to this class.

The two classes deserve different treatment. Invariant features warrant tight likelihoods. Behavior-dependent features warrant deliberately loose ones, because their expected values come from a limited baseline of prior captures. That baseline gives a point estimate of traffic proportions, but little measurement of how those proportions vary across contexts. The fix is to treat the expected proportion as uncertain rather than fixed: model it as a Beta distribution instead of a single value, which yields a Beta-binomial predictive likelihood. The practical effect is that predictions widen by exactly as much as the limited data deserves.

The Feature Catalog

Traffic proportion**.** The count of messages under a candidate opcode versus the proportion recorded at prior patch levels. Behavior-dependent; condition on capture context where possible.

Direction**.** Protocol-invariant by design (the server never trusts the client to report NPC state), so Z2C/C2Z constraints carry large weight (but finite weight), per Cromwell's rule, since a future protocol change or a misparse must remain recoverable.

Size family. Message sizes are discrete and multimodal: $size = base + optional\ blocks$. The empirical size histogram is better evidence than any summary statistic; mean and variance are sufficient only for Gaussian data, which this is not. Compare candidate histograms against the prior patch's directly. Fixed-versus-variable length is a support-size question: Good–Turing estimation infers the unseen probability mass from how many sizes appeared once or twice, which is the rigorous form of "15,404 samples, all size 14 => this is fixed-length." Histogram shape carries structure: fixed strides suggest array counts, discrete modes suggest optional fields. A size below the canonical protocol minimum is near-decisive negative evidence, again at large finite weight.

Layout fit. For the untouched majority under the change model, the prior patch's complete layout is itself a feature: does the old struct decode the collection cleanly, with gates predicting and values plausible throughout? This is cheap, enormous evidence that will confirm most identifications nearly single-handedly. It requires the FieldExtractor to run in a non-asserting TryExtract mode that treats decode failure as data and returning a graded fit report (fraction decoding cleanly, failing gate, offset where reconciliation broke, implausible field) rather than throwing. FailFast remains correct for Glass; for Inference, failure diagnostics are ledger entries. The combination "strong identification evidence, failed layout fit" is a well-defined verdict (identified, mutated) which pre-triages the healing worklist at no extra cost.

Known-plaintext anchors. The database retains validated values durable across runs, e.g. player health, stats, last recorded position (durable across a patch absent a forced zone change). Finding a stored value in a candidate collection is anchoring, analogous to a known-plaintext attack. Its weight is the surprisal of a coincidental match, so high-entropy anchors (an exact position float, ~20+ bits) are near-decisive while low-entropy ones (health = 100 as a byte) are automatically discounted by the same mechanism. Captures' character tags tell Inference which stored values to hunt for. This is the strongest evidence class available and is fully protocol-invariant.

Relative ordering. Formalizes as a Markov model over the per-session opcode stream; the evidence is $P(observed\ neighbors \mid H)$ vs the background. Combat decode was motif detection: Recurring subsequences like StartCast → Cast_Resolution → damage → death. Ordering correlates with frequency (a common opcode neighbors everything), so we use transition probabilities conditioned on occurrence, never raw co-occurrence counts, or evidence is double-counted. Motifs are strongest where the protocol forces a sequence: login and zone-change produce nearly deterministic opcode signatures. This is the highest-weight ordering evidence available.

Burstiness. The dispersion index (variance-to-mean ratio of counts per time window) classifies temporal character on one axis: regular heartbeat < 1, Poisson-like ambient ≈ 1, bursty > 1, state-dependent (combat) bimodal. Rates are behavior-dependent, but the class is a protocol property, stable across captures.

Differential experiments. For opcodes where passive evidence cannot exist (e.g 1-byte payloads), designed experiments manufacture it: Capture under condition X, capture baseline, and the signal is the divergence between the opcode distributions (formally, mutual information between activity and opcode occurrence). This is the active arm of the process; passive scoring handles what the corpus already discriminates.

The Alternative Hypothesis

Every likelihood ratio requires an explicitly stated alternative. ¬H is "some other message type," and its likelihood per feature must come from an empirical background: The pooled distribution over all wire opcodes in the corpus (what a generic size histogram, direction split, or value spread looks like). Uniform or hand-waved backgrounds systematically inflate evidence.

Decision Rule

The accumulate-and-decide structure is Wald's Sequential Probability Ratio Test. Accumulated evidence is compared against two thresholds: cross the upper, accept H and recommend; cross the lower, reject; between them, abstain. The thresholds are not tuning knobs. They derive from tolerable error rates: upper ≈ $\log((1-\beta)/\alpha)$, lower ≈ $\log(\beta/(1-\alpha))$, where α is the acceptable false-identification rate and β the acceptable miss rate. Abstention is a first-class outcome and the formally correct output for undecided evidence; the SPRT's native remedy is more data (such as a longer capture). Operationally, Inference is not expected to work with under 30 minutes of captured data, and handlers run easiest-first to cut noise for later ones.

Combination Rules (standing agreements)

Every feature must state its alternative explicitly. Every feature's weight is capped at a finite maximum (Cromwell operationalized) so no single feature is unrecoverable. Every new feature must declare which existing features it correlates with (ordering ↔ frequency is the known case), with the calibration record as the ongoing audit. Miscalibrated 0.99s mean double-counting before they mean anything else. Assignments are provisional until the full pass completes: an early confident claim is negative evidence for later claimants of the same opcode, but a strong later claim is a conflict that lowers both confidences and flags for review, not a first-come-wins. And runs are deterministic: same capture plus same database yields identical scores, which is what makes stability testing across captures meaningful.

Operating Mode, Calibration, and Improvement

Currently Inference recommends and a human approves; nothing reaches the database unaccepted, so there is no risk in this phase. Recommendations above a threshold (currently 0.5) are recorded with their evidence summaries. The human evaluator supplies gold-standard labels (often readable directly from a hex dump) which enables two distinct checks: whether individual conclusions are right, and whether the confidence machinery is calibrated (claims at 0.99 correct ~99% of the time), assessed with reliability diagrams and

Brier scores across accumulated verdicts. Miscalibration indicts the scoring machinery even when the answers happen to be correct.

Two feedback loops drive improvement. When Inference is wrong, work backwards from the conclusion through the evidence ledger to the failed reasoning. When Inference abstains and the human succeeds at the identification, the human's reasoning is introspected and encoded as a new handler rule. Stability testing (scoring the same hypotheses across many captures of varying size and context) characterizes how performance degrades with data quality. Autonomy expands only as calibration earns it.

Outputs

Accepted conclusions are written to a new PatchLevel; prior levels are never modified and stand as the historical record. Every recommendation carries its itemized deciban ledger, both for review and as the permanent audit trail.

Engineering Assumptions

Inference feeds the extraction machinery untrusted layouts by design, so hostile-input robustness is a requirement, not a nicety: bounded reads, no allocation driven by unvalidated length fields (the inventory-packet buffer-pool deadlock is the standing counterexample), and FailFasts dialed back to graded diagnostics along the Inference path while remaining loud in Glass. External signals such as ShowEQ's published opcode diffs serve as a delayed reference oracle, useful for cross-checking though not authoritative on layout.